

비동기 추적 기반 레이어 스왑 (Async Usage-Tracking Swap) — 기존 연구 조사

날짜

2026-02-26

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

1. 제안 아이디어 요약
2. RTSS 2022 “Demand Layering” 논문과의 비교
 - 2.1 논문 개요
 - 2.2 핵심 기법
 - 2.3 차이점 분석
3. 기존 연구/시스템과의 비교
 - 3.1 DeepSpeed ZeRO-Inference
 - 3.2 FlexGen
 - 3.3 HuggingFace Accelerate
 - 3.4 PIPO (Pipelined Offloading, 2025)
 - 3.5 NEO (MLSys 2025)
 - 3.6 vLLM
 - 3.7 llama.cpp
 - 3.8 Petals
 - 3.9 “Accelerate LLM Inference with Asynchronous Model Offload” (SSDBM 2025)
4. PCIe 대역폭 참고 데이터
 - 4.1 H2D 단방향 대역폭 (본 시나리오에서 사용)
 - 4.2 양방향 대역폭 (참고, 학습 등 D2H 필요 시)
 - 4.3 핵심 결론
5. CUDA 2-Stream 동시 실행 유효성
 - 5.1 하드웨어 요구사항
 - 5.2 2-Way 오버랩
 - 5.3 주의사항
 - 5.4 결론
6. 종합 유효성 평가
 - 6.1 신규성 (Novelty) 평가
 - 6.2 기존 연구와의 핵심 차이점
 - 6.3 가장 유사한 기존 연구 (주의 필요)
 - 6.4 알려진 한계 및 문제점
 - 6.5 최종 결론
7. 참고 문헌 / 출처

비동기 추적 기반 레이어 스왑 (Async Usage-Tracking Swap) — 기존 연구 조사

작성일: 2026-02-26 목적: 제안 아이디어의 유효성 검증 및 기존 연구와의 차별점 분석

1. 제안 아이디어 요약

2026-02-26 수정: 파라미터 오프로딩 추론에서 D2H(GPU→CPU) copy는 불필요하다. CPU에 파라미터 원본이 이미 존재하므로, 사용 완료된 레이어는 GPU 메모리를 free하기만 하면 된다. 따라서 3-stream이 아닌 **2-stream(compute + H2D)** 구조로 수정.

각 transformer 레이어를 비동기로 관리: - 다음에 필요한 레이어를 비동기 H2D로 GPU에 prefetch - 사용 완료된 레이어는 GPU 메모리 해제(free) — D2H copy 불필요 - 2개 CUDA stream (compute, H2D)으로 계산과 전송을 오버랩 - Layer N compute 중 → Layer N+1 H2D prefetch + Layer N-1 GPU free 동시 진행

2. RTSS 2022 “Demand Layering” 논문과의 비교

2.1 논문 개요

- **정식 제목:** “Demand Layering for Real-Time DNN Inference with Minimized Memory Usage”
- **학회:** IEEE 43rd Real-Time Systems Symposium (RTSS 2022)
- **arxiv:** <https://arxiv.org/abs/2210.04024>
- **핵심 접근:** DNN을 레이어 단위로 SSD에서 로드하여 실행, 메모리 사용량을 단일 레이어 수준으로 최소화
- **대상 환경:** iGPU (CPU-GPU 공유 메모리 시스템, 임베디드 시스템)
- **성능:** 96.5% 메모리 감소, 14.8% 지연 오버헤드

2.2 핵심 기법

항목	Demand Layering (RTSS 2022)	제안 아이디어
메모리 계층	SSD ↔ GPU (iGPU 공유메모리)	CPU RAM ↔ GPU VRAM (dGPU)
파이프라이닝	O (로딩과 실행 오버랩)	O (3-stream 파이프라인)
비동기 프리페치	파이프라인 구조로 은닉	명시적 CUDA stream 기반 비동기
사용 후 처리	SSD writeback	GPU free (D2H 불필요, CPU에 원본 보유)
PCIe 활용	해당 없음 (SSD 단방향)	H2D 단방향 (prefetch만)
대상 모델	일반 DNN (CNN 등)	Transformer (LLM/VLM)
실시간 보장	O (WCET 분석 포함)	해당 없음 (처리량 최적화)

2.3 차이점 분석

Demand Layering은 **iGPU 임베디드 환경**에서 **SSD-GPU 간** 레이어 로딩에 초점을 맞춘 반면, 제안 아이디어는 **dGPU 환경**에서 **CPU-GPU 간 PCIe** 양방향 동시 전송을 활용한다는 점에서 근본적으로 다른 하드웨어 타깃이다. 또한 Demand Layering은 **실시간성 보장(WCET 분석)**이 핵심이고, 제안 아이디어는 **처리량 최적화(throughput)**가 목표이다.

결론: 레이어 단위 로딩 + 파이프라이닝이라는 고수준 개념은 유사하지만, 하드웨어 타깃, 전송 방식(SSD vs PCIe 양방향), 최적화 목표(실시간 vs 처리량)가 모두 다르다.

3. 기존 연구/시스템과의 비교

3.1 DeepSpeed ZeRO-Inference

- **구현 방식:** 모델 가중치를 CPU/NVMe로 완전 오프로딩, GPU 메모리는 activation 저장에 활용
- **비동기 프리페치:** 사용함. 다음 레이어를 미리 fetch하여 현재 레이어 연산과 오버랩
- **다중 GPU 최적화:** 각 GPU가 레이어의 일부만 fetch → 집합 PCIe 대역폭 활용
- **비동기 all-gather:** 한 레이어 연산 중 다른 레이어 통신 병렬 수행

항목	DeepSpeed ZeRO-Inference	제안 아이디어
프리페치	O (다음 레이어 미리 fetch)	O (N+2 레이어 H2D)
사용 후 처리	메모리 해제	GPU free (D2H 불필요)
PCIe 활용	주로 H2D 단방향	H2D 단방향 (prefetch)
CUDA stream 수	2개 (compute + copy) 수준	2개 (compute + H2D)

유사점: 두 방식 모두 H2D prefetch에 집중하고, 사용 후 GPU 메모리를 해제하는 구조. 파라미터 원본이 CPU에 유지되므로 D2H copy는 양쪽 다 불필요.

3.2 FlexGen

- **설계 목표:** 단일 GPU에서 175B 모델 추론 (처리량 최적화)
- **메모리 계층:** GPU ↔ CPU ↔ Disk 3계층 오프로딩
- **핵심 기법:** 선형 프로그래밍(LP)으로 최적 텐서 배치/접근 패턴 탐색
- **I/O 최적화:** 다중 CUDA stream + CPU thread로 I/O와 compute 오버랩
- **파이프라인:** 배치 레벨 파이프라인 병렬처리 (super-linear scaling 가능)

항목	FlexGen	제안 아이디어
오프로딩 계층	3계층 (GPU-CPU-Disk)	2계층 (GPU-CPU)
스케줄링	LP 기반 최적 스케줄링	Used 플래그 기반 동적
파이프라인 단위	배치 레벨	레이어 레벨
PCIe 활용	다중 stream (H2D/D2H)	2-stream (H2D + compute)

차별점: FlexGen은 배치 처리(throughput) 최적화와 LP 기반 정적 스케줄링이 핵심이고, 제안 아이디어는 단일 요청의 레이어 단위 동적 스왑에 초점이 맞춰져 있다.

3.3 HuggingFace Accelerate

- **cpu_offload / disk_offload:** forward hook으로 레이어 실행 직전 GPU 로드, 직후 CPU/Disk 반환
- **비동기 전송:** 최근 추가/개발 중. `non_blocking=True` 옵션, CUDA copy_stream 활용
- **프리페치:** 다음 레이어를 별도 CUDA stream으로 미리 로드 (Layer prefetching with streams)

항목	HF Accelerate	제안 아이디어
Hook 기반	O (pre/post forward hooks)	유사 (used 플래그)
비동기 전송	최근 도입 (copy_stream)	3-stream 설계
사용 후 처리	비동기 offload 옵션	GPU free (D2H 불필요)
PCIe 활용	H2D prefetch (최근 도입)	H2D prefetch (2-stream)

차별점: Accelerate는 범용 프레임워크로 순차적 offload가 기본이고 비동기는 최근 추가된 옵션이다. 제안 아이디어는 2-stream H2D prefetch + GPU free가 설계의 핵심이다. 기능적으로 유사한 방향으로 수렴 중이다.

참고: HuggingFace Accelerate Issue #3267에서 “hooks with overlapped transfers and computations”가 논의되고 있어, 커뮤니티에서도 동일한 방향의 최적화를 탐색 중이다.

3.4 PIPO (Pipelined Offloading, 2025)

- **arxiv:** <https://arxiv.org/abs/2504.03664>
- **핵심:** GPU-CPU-Disk 간 세밀한 파이프라인 오프로딩
- **4가지 태스크:** computation, weight loading, KV-cache loading, KV-cache saving
- **오버랩 전략:** Layer N+1의 weight loading과 Layer N의 computation을 동시 수행
- **성능:** GPU 활용률 40% → 90%+, 최대 3.1x 처리량 향상

항목	PIPO	제안 아이디어
파이프라인 세분화	4종 태스크 (weight, KV-load, KV-save, compute)	2-stream (compute, H2D) + free
스케줄링	Thread pool + task queue (동적)	순차 prefetch + free (동적)
자동 설정	HW 스펙 기반 자동 최적화	수동 설정
양자화 커널	INT4 커스텀 커널 포함	없음 (FP16/BF16 유지)

차별점: PIPO는 제안 아이디어와 가장 유사한 기존 연구이다. 핵심 차이는 PIPO가 KV-cache saving(D2H)을 포함하여 4종 태스크를 관리하는 반면, 제안 아이디어는 파라미터 원본이 CPU에 유지되므로 D2H가 불필요하여 2-stream(compute + H2D) + free로 단순화된다는 점이다. 핵심 개념(파이프라인 오프로딩으로 compute/transfer 오버랩)은 동일하다.

3.5 NEO (MLSys 2025)

- **논문:** “NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference”
- **핵심:** attention 연산 일부 + KV cache를 CPU로 오프로딩
- **비동기 파이프라인:** 레이어 단위 스와핑(layer-wise swapping) 으로 KV 전송과 연산 오버랩
- **비대칭 파이프라이닝:** GPU/CPU 부하를 비대칭적으로 분배

항목	NEO	제안 아이디어
오프로딩 대상	attention + KV cache	전체 레이어 가중치
레이어 단위 스왑	O (layer-wise swapping)	O (used 플래그 기반)
PCIe 오버랩	O (즉시 전송 시작)	O (3-stream 동시)
연산 분할	CPU에서 일부 연산 수행	GPU only

차별점: NEO는 가중치 오프로딩이 아닌 KV cache + attention 오프로딩에 초점이다. 연산 자체를 CPU/GPU에 분할 수행한다는 점에서 방향이 다르다.

3.6 vLLM

- **PagedAttention:** KV cache를 비연속 페이지로 관리 (OS 가상 메모리와 유사)
- **CPU Offloading:** 가중치를 CPU pinned memory에 두고, 레이어 실행 시 GPU 로드 → 완료 후 GPU 메모리 해제
- **KV cache offloading:** 2026년 1월 vLLM 0.11.0에서 KV cache를 CPU로 오프로딩하는 기능 추가
- **CUDA 커널:** 효율적인 per-layer KV cache 스왑 커널 구현

차별점: vLLM의 CPU offload는 제안 아이디어와 매우 유사한 레이어 단위 로드/언로드를 수행하지만, 서빙(다중 요청) 최적화가 주 목표이다. PagedAttention은 KV cache 관리의 혁신이지 가중치 스왑과는 다른 문제이다.

3.7 llama.cpp

- **GPU offload:** `--gpu-layers` 파라미터로 레이어 수준 GPU/CPU 분배
- **mmap:** 디스크에서 필요한 부분만 메모리 매핑 (전체 RAM 로딩 불필요)
- **MoE 최적화:** expert weight를 RAM에 두고 activation만 PCIe로 전송 (가중치 이동 대신)

차별점: llama.cpp는 정적 레이어 배치(N개 레이어를 GPU에, 나머지 CPU)를 사용하며, 런타임에 동적 스왑을 하지 않는다. MoE 특화 최적화(activation 전송)는 독특하나 dense 모델에는 적용 불가.

3.8 Petals

- 접근: BitTorrent 방식 P2P 분산 추론. 각 서버가 모델의 일부 레이어를 보유
- 통신: 네트워크를 통한 레이어 간 activation 전달

차별점: Petals는 네트워크 분산 환경이므로 단일 머신 PCIe 최적화와는 완전히 다른 문제이다.

3.9 “Accelerate LLM Inference with Asynchronous Model Offload” (SSDBM 2025)

- 저자: Jie Ye, Anthony Kougkas, Xian-He Sun, Bogdan Nicolae
- 핵심: vLLM의 동기식 offload를 비동기식으로 개선
- 성능: vLLM 대비 1.1x+ 처리량 향상, TTFT/TPOT 감소

가장 최근(2025) 연구로, 비동기 오프로딩의 실효성을 vLLM 위에서 검증한 사례.

4. PCIe 대역폭 참고 데이터

2026-02-26 수정: 파라미터 오프로딩 추론에서는 H2D(prefetch)만 사용하고 D2H는 불필요하므로, 양방향 동시 전송의 대역폭 감소 문제는 본 시나리오에 해당하지 않는다. 아래 데이터는 참고용으로 유지한다.

4.1 H2D 단방향 대역폭 (본 시나리오에서 사용)

환경	H2D (pinned)	비고
PCIe 3.0 (단일 전송)	~8.5 GB/s	기본
PCIe 3.0 (2-8MB 청크)	~ 12.4 GB/s	최적 구간
PCIe 4.0 A100	~24.8 GB/s	고급 GPU

4.2 양방향 대역폭 (참고, 학습 등 D2H 필요 시)

환경	단방향 H2D	단방향 D2H	양방향 H2D	H2D 감소율
PCIe 3.0 RTX 3080	~12 GB/s	~12 GB/s	~9.45 GB/s	-21%
PCIe 4.0 A100	~24.8 GB/s	~25.9 GB/s	~11 GB/s	-56%

4.3 핵심 결론

파라미터 오프로딩 추론은 H2D만 사용하므로, 양방향 대역폭 경합 문제가 발생하지 않는다.

PCIe 대역폭을 단방향으로 온전히 활용할 수 있어, 이전에 우려했던 21-56% 성능 저하는 해당 없다.

5. CUDA 2-Stream 동시 실행 유효성

5.1 하드웨어 요구사항

- Compute Capability ≥ 2.0 + `asyncEngineCount` ≥ 1 필요
- 현대 GPU (Kepler 이후)는 모두 지원
- 2-stream(compute + H2D)은 DMA engine 1개만으로 충분

5.2 2-Way 오버랩

- Stream A: H2D transfer (DMA engine)
- Stream B: Kernel execution (SM)
- 서로 다른 하드웨어 유닛을 사용하므로 동시 실행 가능
- D2H 스트림이 없으므로 DMA engine 경합 없음

5.3 주의사항

1. 호스트 메모리는 반드시 **pinned memory**여야 함 (pageable memory $\rightarrow$ async 불가능)
2. 전송 크기가 너무 작으면 오버헤드가 전송 시간을 초과
3. **CUDA event** 기반 동기화 필요 (stream 간 의존성 관리)
4. **WSL2 환경**에서는 `cudaMemPrefetchAsync`(CPU) 미지원 등 제약 있음

5.4 결론

2개 CUDA stream(compute + H2D)의 동시 실행은 하드웨어적으로 완전히 지원된다. D2H가 없으므로 PCIe 양방향 대역폭 경합 문제도 발생하지 않아, H2D 대역폭을 온전히 활용할 수 있다.

6. 종합 유효성 평가

6.1 신규성 (Novelty) 평가

구성 요소	신규성	비고
레이어 단위 오프로딩	낮음	DeepSpeed, FlexGen, Accelerate 등 광범위하게 사용
비동기 H2D 프리페치	낮음	DeepSpeed, PIPO, vLLM, Accelerate에서 이미 구현
D2H 불필요 인식 (GPU free)	중간	대부분의 기존 연구는 D2H를 기본 포함하나, 파라미터 추론에서는 불필요
2 CUDA stream (compute+H2D) + free	낮음	compute + prefetch 오버랩은 표준 기법
Pinned memory 기반 H2D 최적화	중간	2-8MB 청크 최적화 + pinned memory 조합은 체계적 분석 사례 적음

6.2 기존 연구와의 핵심 차이점

- D2H 제거:** 파라미터 오프로딩 추론에서 D2H가 불필요함을 인식하고, GPU free만으로 VRAM을 확보. 기존 연구 대부분은 D2H를 포함하는 3-stream 설계를 사용
- PCIe 단방향 전용 활용:** D2H가 없으므로 양방향 대역폭 경합 없이 H2D 대역폭을 온전히 활용
- 단순하고 명확한 설계:** PIPO나 FlexGen 대비 구현이 간결 (2-stream + pinned memory + free)

6.3 가장 유사한 기존 연구 (주의 필요)

PIPO (2025)가 가장 유사하다. PIPO는: - GPU-CPU-Disk 간 파이프라인 오프로딩 - Weight loading, KV-cache loading, KV-cache saving, computation을 동시 수행 - Thread pool 기반 동적 스케줄링 - GPU 활용률 40% → 90%

제안 아이디어가 PIPO와 차별화되는 점: - 파라미터 추론에서 D2H 불필요 → 2-stream으로 단순화 (PIPO는 KV-save용 D2H 포함) - Alpamayo와 같은 특정 VLM 파이프라인 (Vision→VLM→Expert)에서의 모듈 간 전환 최적화 - Pinned memory + 최적 청크(2-8MB) 조합으로 H2D 대역폭 극대화

6.4 알려진 한계 및 문제점

1. **PCIe가 병목:** 레이어 전송 시간(~45ms H2D)이 레이어 실행 시간(~0.5ms)의 90배 → 파이프라이닝으로 완전 은닉 불가능
2. **Pinned memory 제약:** 전체 모델을 pinned로 올릴 수 없음 → 부분 pinned 버퍼 필요 (System RAM 16GB 한계)
3. **WSL2 오버헤드:** per-transfer 고정 오버헤드 7배
4. **Transformer 단일 배치 특성:** batch_size=1에서 레이어 연산 시간이 매우 짧아, 전송을 은닉할 연산 시간이 부족
5. ~~PCIe 양방향 대역폭 감소~~: D2H가 불필요하므로 이 문제는 해당 없음

6.5 최종 결론

제안 아이디어의 핵심 구성 요소들은 대부분 기존 연구에서 이미 사용되고 있다. 특히: - 레이어 단위 오프로딩: DeepSpeed, FlexGen, vLLM, Accelerate - 비동기 프리페치: DeepSpeed, PIPO, NEO - CUDA stream 기반 오버랩: 표준 CUDA 기법

그러나 다음 측면에서 연구 기여의 여지가 있다: 1. D2H 제거의 실효성 검증: 파라미터 추론에서 D2H를 제거하고 GPU free만으로 충분함을 실증 2. 2-stream + pinned memory + 최적 청크의 체계적 조합 분석 3. 특정 도메인(자율주행 VLM)에서의 모듈+레이어 복합 스왑 최적화 4. 제한된 VRAM(12GB)에서의 실용적 시스템 구현 및 성능 분석

7. 참고 문헌 / 출처

1. Demand Layering (RTSS 2022): <https://arxiv.org/abs/2210.04024>
2. DeepSpeed ZeRO-Inference: <https://www.deepspeed.ai/2022/09/09/zero-inference.html>

3. FlexGen: <https://arxiv.org/abs/2303.06865>
4. HuggingFace Accelerate Big Modeling: https://huggingface.co/docs/accelerate/en/package_reference/big_modeling
5. HuggingFace Accelerate Issue #3267 (overlapped hooks): <https://github.com/huggingface/accelerate/issues/3267>
6. PIPO: <https://arxiv.org/abs/2504.03664>
7. NEO (MLSys 2025): <https://arxiv.org/abs/2411.01142>
8. vLLM KV Offloading: <https://blog.vllm.ai/2026/01/08/kv-offloading-connector.html>
9. vLLM CPU Offloading: <https://github.com/vllm-project/vllm/issues/3563>
10. Petals: <https://arxiv.org/abs/2209.01188>
11. llama.cpp: <https://github.com/ggml-org/llama.cpp>
12. NVIDIA CUDA Overlap Transfers: <https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/>
13. PCIe Bidirectional Bandwidth Asymmetry: <https://forums.developer.nvidia.com/t/asymmetric-pcie-bandwidth-in-bidirectional-transfers-h2d-drops-56-while-d2h-maintains-performance/352186>
14. PCIe Bandwidth Contention: <https://forums.developer.nvidia.com/t/bandwidth-contention-of-concurrent-h2d-d2h-memory-copy/250140>
15. Asynchronous Model Offload (SSDBM 2025): <https://ssdbm.org/2025/assets/poster/8884-Jie.pdf>
16. HeadInfer (Head-wise Offloading): <https://arxiv.org/abs/2502.12574>